

Diagrama de Casos de Uso

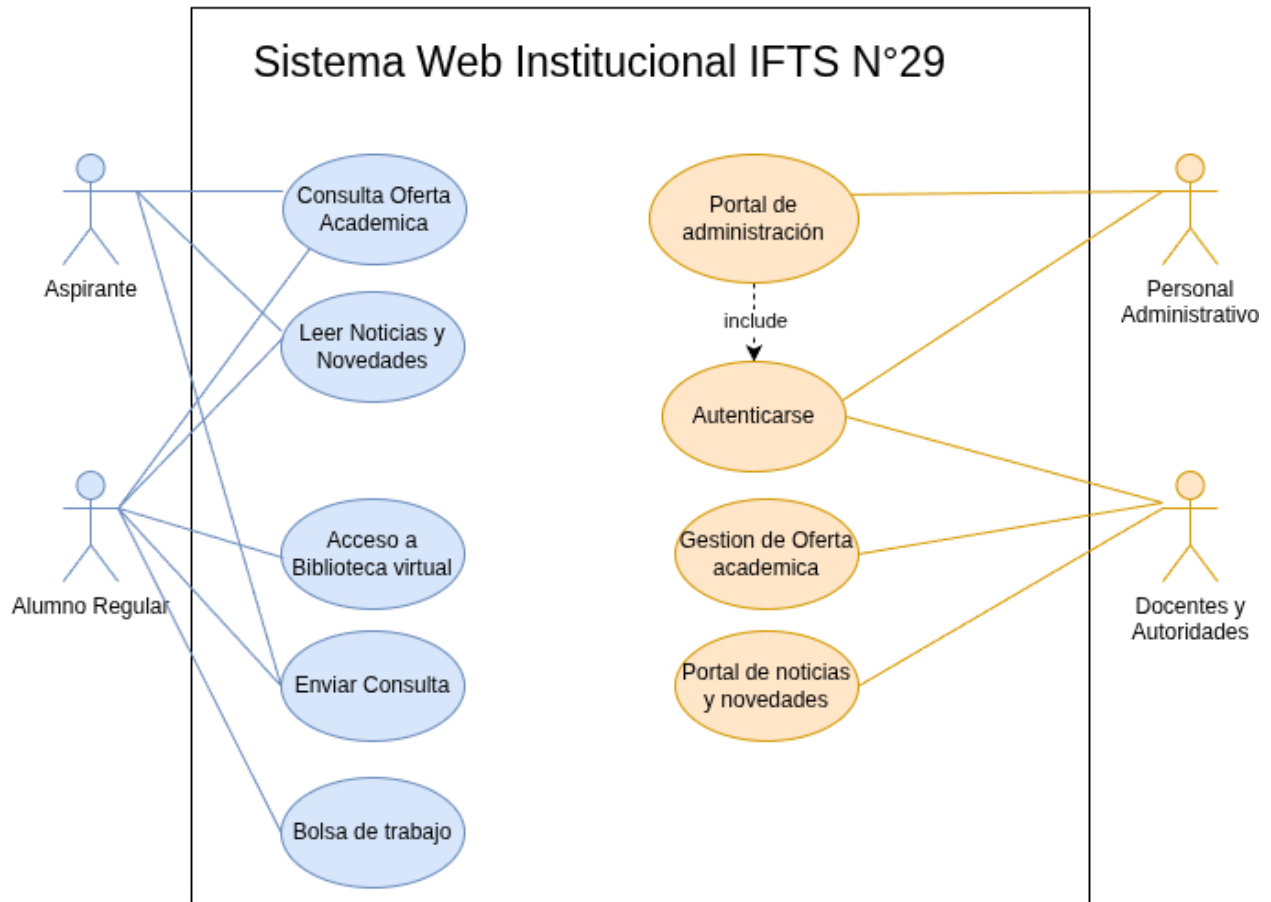


Diagrama de Clases

```

classDiagram
    class Usuario {
        +int id
        +String nombre
        +String email
        +String password
        +Date fecha_registro
        +login()
        +logout()
    }
    class Asistente
    class Docente
    class Autoridad
    class Alumno {
        +String matricula
        +inscribirse()
    }
    class Empresa {
        +String cuit
        +String razon_social
        +publicarOferta()
    }
    class OfertaLaboral {
        +int id
        +String titulo
        +String descripcion
        +String requisitos
        +Date fecha_cierre
    }
    class Consultor {
        +int id
        +String remision_numero
        +String remision_email
        +String mensaje
        +Date fecha_envio
        +String estado_resolucion
    }
    class CategoriaPro {
    }
    class Administrativo {
        +String cargo
        +gestionarContenido()
    }
    class Publicacion {
        +int id
        +String titulo
        +String cuerpo
        +Date fecha_publicacion
        +String estado
    }
    class Evento {
        +Date fecha_evento
        +String lugar
    }
    class Comunicado
    class Categoria {
    }
    class Tecnatura {
        +int id
        +String nombre
        +int duracion_anios
        +String descripcion
    }
    class PlanEstudio
    class PerfilProfesional
    class RecursoAcademico
    class Exhurneo {
    }
    class Testimonio
    class Publicacion {
        +Date fecha_publicacion
        +String estado_publicacion
    }
    class Mensaje {
    }
    class DocumentoOficial {
    }
    class Noticia {
    }

    Usuario <|-- Asistente
    Usuario <|-- Docente
    Usuario <|-- Autoridad
    Usuario <|-- Alumno
    Usuario <|-- Empresa
    Usuario <|-- Consultor
    Usuario <|-- Administrativo
    Usuario <|-- Publicacion
    Usuario <|-- Evento
    Usuario <|-- Comunicado

    Alumno --> Tecnatura : cursa
    Alumno --> Exhurneo : brinda
    Alumno --> Publicacion : realiza
    Empresa --> OfertaLaboral : publica
    Consultor --> Publicacion : genera
    Administrativo --> Publicacion : sube
    Publicacion <|-- Evento
    Publicacion <|-- Comunicado
    Publicacion --> Categoria : clasifica
    Tecnatura --> PlanEstudio : tiene
    Tecnatura --> PerfilProfesional : tiene
    Tecnatura --> RecursoAcademico : contiene
    Exhurneo --> Testimonio : brinda
    Exhurneo --> Publicacion : realiza
    Publicacion --> Mensaje : coordina
    Publicacion --> DocumentoOficial : genera
    Publicacion --> Noticia : genera
  
```

The diagram illustrates the relationships between various entities in a system. The entities and their attributes are as follows:

- Usuario**: +int id, +String nombre, +String email, +String password, +Date fecha_registro, +login(), +logout().
- Asistente**: (Specialization of Usuario)
- Docente**: (Specialization of Usuario)
- Autoridad**: (Specialization of Usuario)
- Alumno**: +String matricula, +inscribirse().
- Empresa**: +String cuit, +String razon_social, +publicarOferta().
- OfertaLaboral**: +int id, +String titulo, +String descripcion, +String requisitos, +Date fecha_cierre.
- Consultor**: +int id, +String remision_numero, +String remision_email, +String mensaje, +Date fecha_envio, +String estado_resolucion.
- CategoriaPro**: (Generalization of Consultor)
- Administrativo**: +String cargo, +gestionarContenido().
- Publicacion**: +int id, +String titulo, +String cuerpo, +Date fecha_publicacion, +String estado.
- Evento**: +Date fecha_evento, +String lugar.
- Comunicado**: (Specialization of Publicacion)
- Categoria**: (Generalization of Publicacion)
- Tecnatura**: +int id, +String nombre, +int duracion_anios, +String descripcion.
- PlanEstudio**: (Specialization of Tecnatura)
- PerfilProfesional**: (Specialization of Tecnatura)
- RecursoAcademico**: (Specialization of Tecnatura)
- Exhurneo**: (Generalization of Tecnatura)
- Testimonio**: (Specialization of Exhurneo)
- Publicacion**: +Date fecha_publicacion, +String estado_publicacion.
- Mensaje**: (Generalization of Publicacion)
- DocumentoOficial**: (Specialization of Mensaje)
- Noticia**: (Specialization of Mensaje)

The relationships between the entities are as follows:

- Usuario** is a generalization of **Asistente**, **Docente**, **Autoridad**, **Alumno**, **Empresa**, **Consultor**, **Administrativo**, **Publicacion**, **Evento**, and **Comunicado**.
- Alumno** *cursa* **Tecnatura**.
- Alumno** *brinda* **Exhurneo**.
- Alumno** *realiza* **Publicacion**.
- Empresa** *publica* **OfertaLaboral**.
- Consultor** *genera* **Publicacion**.
- Administrativo** *sube* **Publicacion**.
- Publicacion** is a generalization of **Evento** and **Comunicado**.
- Publicacion** *clasifica* **Categoria**.
- Tecnatura** *tiene* **PlanEstudio** and **PerfilProfesional**.
- Tecnatura** *contiene* **RecursoAcademico**.
- Exhurneo** *brinda* **Testimonio**.
- Exhurneo** *realiza* **Publicacion**.
- Publicacion** *coordina* **Mensaje**.
- Publicacion** *genera* **DocumentoOficial** and **Noticia**.

Diagrama de Secuencia

```
sequenceDiagram
    actor Admin as Personal Administrativo
    participant CMS as Sanity Studio (CMS)
    participant API as Sanity Content Lake (API)
    participant Frontend as Frontend (Next.js / Vercel)
    actor Student as Aspirante / Alumno

    Note over Admin, CMS: Fase 1: Gestión de Contenido (CRUD)
    Admin->>CMS: 1 Inicia sesión con credenciales
    CMS-->>Admin: 2 Acceso concedido al panel
    Admin->>CMS: 3 Redacta nueva Noticia y presiona "Publish"
    CMS->>API: 4 Mutación de datos (JSON)
    API-->>CMS: 5 Documento guardado y publicado
    CMS-->>Admin: 6 Confirmación visual de éxito

    Note over Frontend, API: Fase 2: Sincronización
    Frontend->>API: 7 Webhook notifica cambios (Revalidación de cache)

    Note over Frontend, Student: Fase 3: Experiencia de Usuario (UX)
    Student->>Frontend: 8 Navega al Portal de Noticias
    Frontend->>API: 9 Ejecuta consulta GROQ para obtener noticias
    API-->>Frontend: 10 Retorna datos de la nueva Noticia (JSON)
    Frontend->>Frontend: 11 Renderiza los componentes de React con Tailwind
    Frontend->>Student: 12 Muestra la interfaz actualizada en menos de 2s
```

The diagram illustrates the sequence of interactions between five participants: Personal Administrativo, Sanity Studio (CMS), Sanity Content Lake (API), Frontend (Next.js / Vercel), and Aspirante / Alumno. The process is divided into three main phases:

- Fase 1: Gestión de Contenido (CRUD)**:
 - 1. Personal Administrativo sends "Inicia sesión con credenciales" to Sanity Studio (CMS).
 - 2. Sanity Studio (CMS) returns "Acceso concedido al panel" to Personal Administrativo.
 - 3. Personal Administrativo sends "Redacta nueva Noticia y presiona 'Publish'" to Sanity Studio (CMS).
 - 4. Sanity Studio (CMS) sends "Mutación de datos (JSON)" to Sanity Content Lake (API).
 - 5. Sanity Content Lake (API) returns "Documento guardado y publicado" to Sanity Studio (CMS).
 - 6. Sanity Studio (CMS) returns "Confirmación visual de éxito" to Personal Administrativo.
- Fase 2: Sincronización**:
 - 7. Frontend (Next.js / Vercel) sends "Webhook notifica cambios (Revalidación de cache)" to Sanity Content Lake (API).
- Fase 3: Experiencia de Usuario (UX)**:
 - 8. Aspirante / Alumno sends "Navega al Portal de Noticias" to Frontend (Next.js / Vercel).
 - 9. Frontend (Next.js / Vercel) sends "Ejecuta consulta GROQ para obtener noticias" to Sanity Content Lake (API).
 - 10. Sanity Content Lake (API) returns "Retorna datos de la nueva Noticia (JSON)" to Frontend (Next.js / Vercel).
 - 11. Frontend (Next.js / Vercel) performs a self-action: "Renderiza los componentes de React con Tailwind".
 - 12. Frontend (Next.js / Vercel) sends "Muestra la interfaz actualizada en menos de 2s" to Aspirante / Alumno.

Diagrama de Actividades

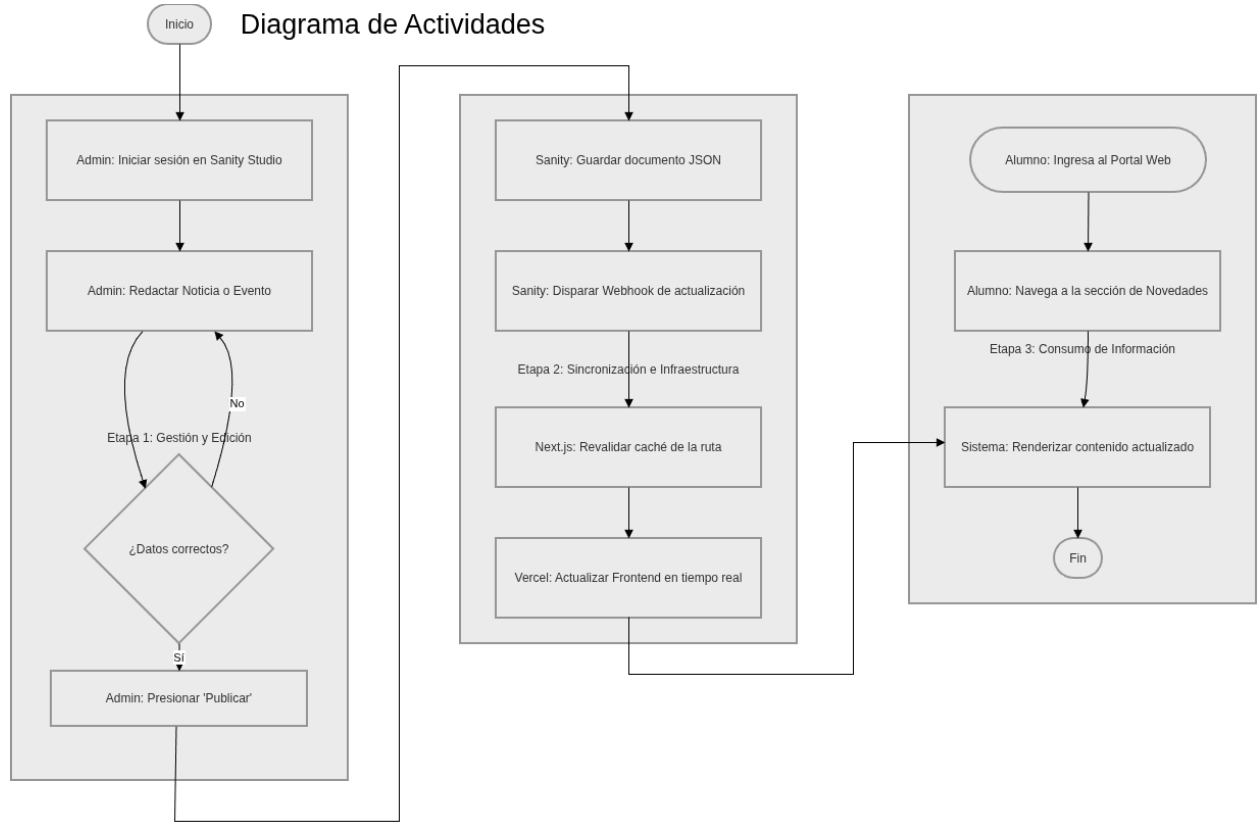


Diagrama de Estados

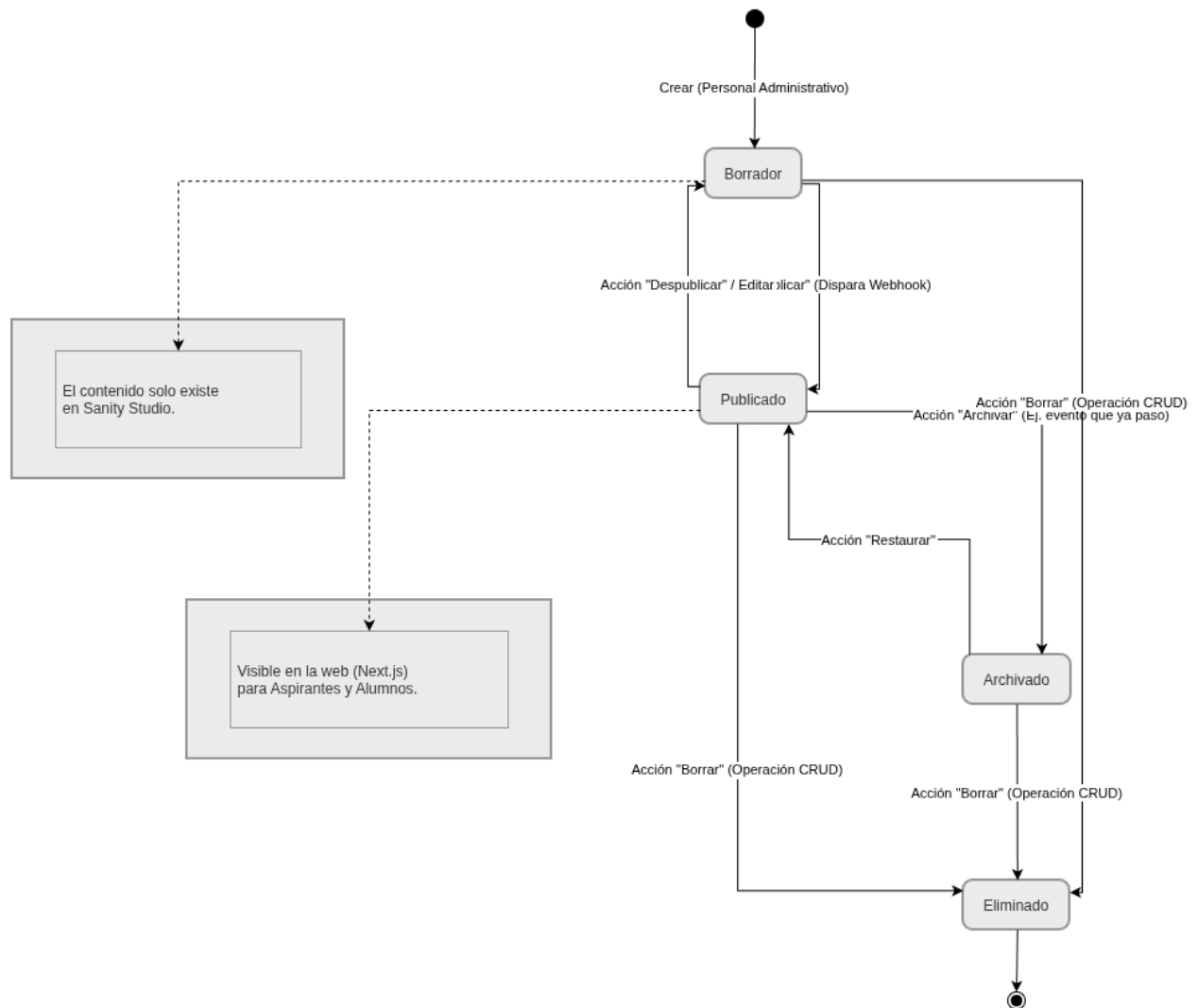


Diagrama de Componentes

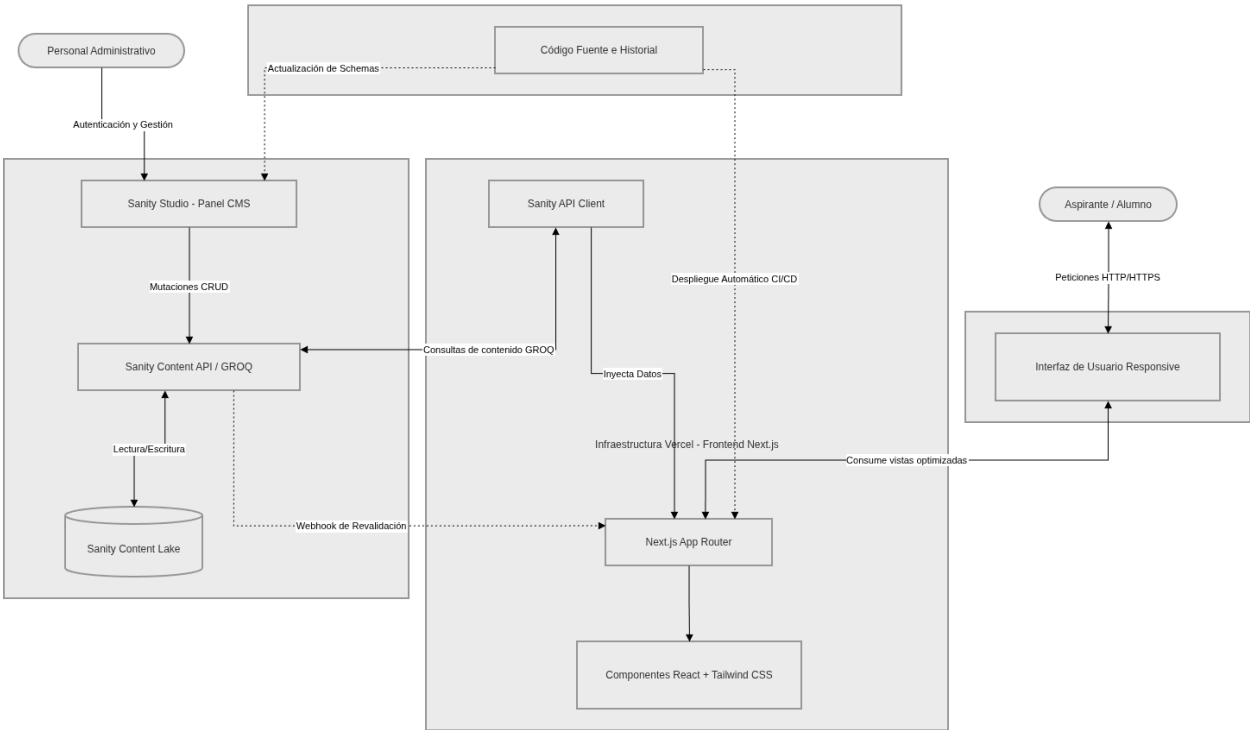


Diagrama de Despliegue

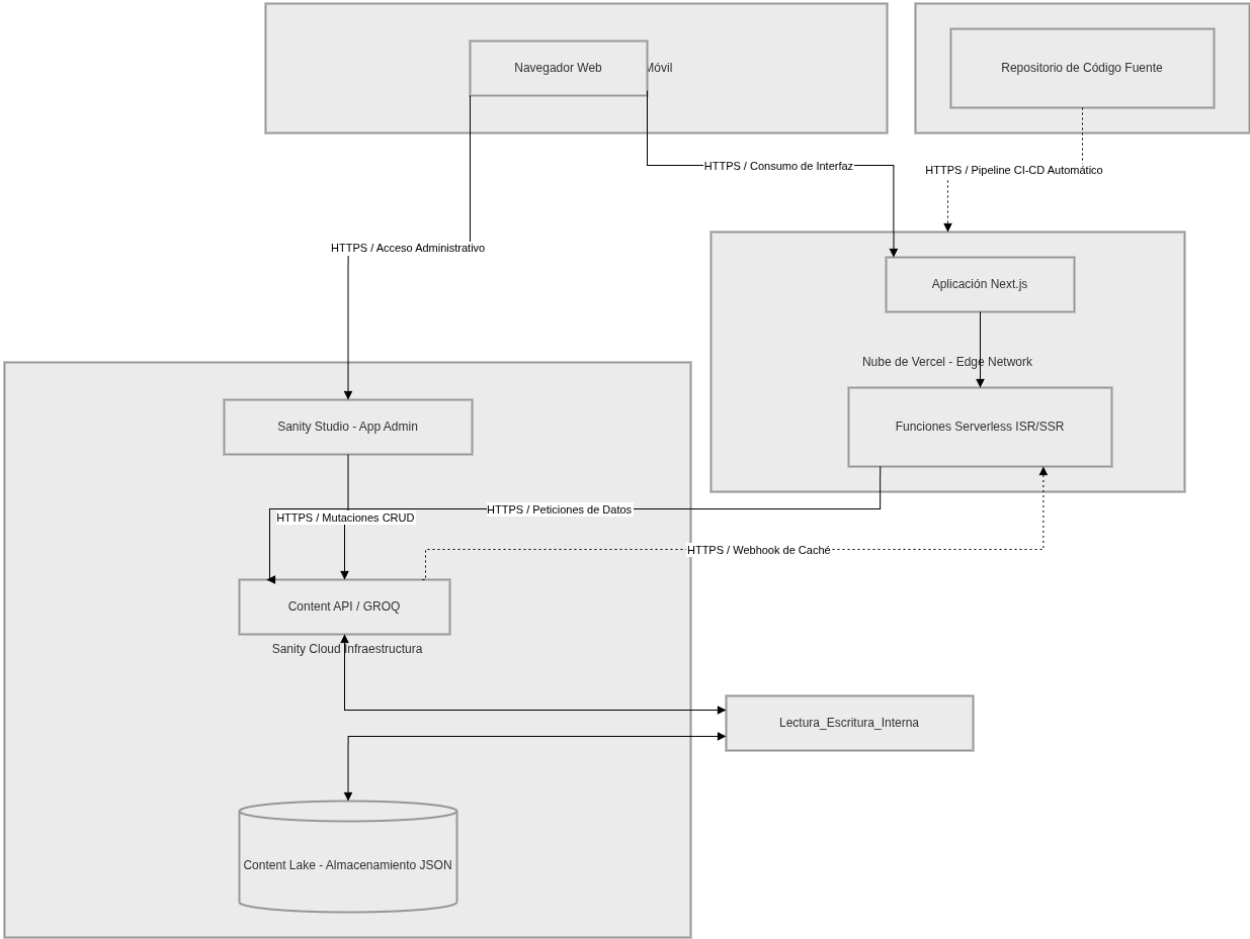


Diagrama Entidad-Relación

